BUILDTWIN · ENGINEERING

Full-Stack Engineer

Take-Home Assignment

Build **Attendance Gaps**, a multi-tenant web app with single sign-on and role-based access. Then explain how you would run it for hundreds of companies at low latency and sensible cost.

TIME	YOU SUBMIT
About 7 hours of focused work. Please stop at 8, even if you haven't finished.	A git repository with code, tests, `README.md`, `DESIGN.md` and `NOTES.md`.
TOOLS	**NEXT STEP**
Your usual editor and AI assistants are welcome. Tell us how you used them.	A 75-minute live session: demo, code walkthrough, a small live change, debugging and a design discussion.

CONTENTS

1. The problem
2. What to deliver
3. Stack and ground rules
4. Companies, sign-in and access
5. Data
6. Rules
7. Part A: build
8. Part B: DESIGN.md
9. Part C: NOTES.md and your repository
10. Optional extras
11. How we evaluate
12. The live session
13. Submitting and questions

1 The problem

Construction companies pay their staff monthly, based on *payable days* worked out from daily attendance records. When nobody records attendance for a working day, the gap goes unnoticed until payday, and people are paid for days nobody accounted for.

Build **Attendance Gaps**, a multi-tenant web app. Each company signs in through its own identity provider, and people can, depending on their role:

- see, per employee and month, which scheduled working days have no attendance record;
- fill those gaps;
- see payable days worked out under their company's policy.

No construction or payroll knowledge is needed. Every rule you need is in this document.

2 What to deliver

PART	WHAT	TIME
A. Build	A working backend, database, single sign-on, role-based access and Angular frontend, with tests and performance measurements	about 5.5 h
B. Design	`DESIGN.md`: how the system runs at scale on AWS, what it costs, and where the latency goes	about 1 h
C. Notes	`NOTES.md`: your decisions, measurements, a browser-storage inventory and an AI-use note	about 30 min

> We value judgment and correctness over completeness. If you run short of time, finish fewer things well and tell us what you would do next.

3 Stack and ground rules

- **Backend:** Node.js 20 or later, JavaScript or TypeScript. Write each endpoint as an **AWS Lambda handler** (an API Gateway event in, a response out). Run the handlers locally with any adapter you like: a small Express wrapper, `sam local start-api`, or serverless-offline.
- **Database:** PostgreSQL in Docker, with migrations and a seed script.
- **Identity:** an OpenID Connect provider running locally in Docker, with a separate issuer for each company. For example: Keycloak (one realm per company), Dex, or `ghcr.io/navikt/mock-oauth2-server`, which serves any number of issuers without configuration.
- **Frontend:** Angular 14 or later.
- Everything runs locally. No cloud account is needed or allowed, and nothing may cost money.
- Use established libraries for OIDC and JWT validation; don't hand-roll cryptography.
- A reviewer should have your app running within 10 minutes, using only your README.

4 Companies, sign-in and access

4.1 Single sign-on

- Each company has its own OIDC issuer. A user starts by entering their work email. The app finds the company from the email's domain and sends the user to that company's issuer, using the Authorization Code flow with PKCE.
- The API accepts a token only if all of these hold: the issuer configured for the caller's company issued it, it is meant for this app, it has not expired, and its email is in that company's domain.
- A signed-in identity is matched to an employee by email. Unknown emails, and people whose `left_on` date has passed, are refused even if the identity provider let them sign in.
- Signing out ends the app session for that company.

4.2 Roles and permissions

PERMISSIONS

PERMISSION	ALLOWS
`summary:read:self`	Seeing your own row
`summary:read:reports`	Seeing your direct reports' rows
`summary:read:company`	Seeing everyone in your company
`attendance:write:reports`	Recording attendance for your direct reports
`attendance:write:company`	Recording attendance for anyone in your company

ROLES

ROLE	AVAILABLE TO	PERMISSIONS
`EMPLOYEE`	Every company	`summary:read:self`
`MANAGER`	Every company	`summary:read:self`, `summary:read:reports`, `attendance:write:reports`
`HR_ADMIN`	Every company	`summary:read:company`, `attendance:write:company`
`PAYROLL_VIEWER`	Birch Infra only	`summary:read:company`

- A user can see everyone covered by their role's read permissions combined, and never anyone in another company. An employee outside that set is reported as not found (404).
- Trying to record attendance for someone you can see, but may not record for, returns 403.
- Roles and their permissions are data. Adding a role, or changing what a role may do, must not need a code change or a redeploy.
- The frontend may hide what a user can't do, but the API is the authority.

5 Data

Load this data with your seed script. Every company works in the Asia/Kolkata timezone.

COMPANIES

```
id,name,email_domain,deduct_unaccounted
C1,Alder Works,alder.example,false
C2,Birch Infra,birch.example,true
```

REGIONAL HOLIDAYS

```
region,company_id,date
NORTH,C1,2026-08-15
NORTH,C1,2026-08-26
SOUTH,C1,2026-08-28
COAST,C2,2026-08-12
```

EMPLOYEES (EVERY EMPLOYEE IS ALSO A USER OF THE APP)

```
id,company_id,name,email,region,roster,joined_on,left_on,manager_id,role
E01,C1,Asha,asha@alder.example,NORTH,MON-FRI,2024-01-10,,M01,EMPLOYEE
E02,C1,Ben,ben@alder.example,NORTH,MON-SAT,2023-05-01,,M01,EMPLOYEE
E03,C1,Chitra,chitra@alder.example,SOUTH,MON-FRI,2026-08-17,,M02,EMPLOYEE
E04,C1,Dev,dev@alder.example,SOUTH,MON-FRI,2022-02-01,2026-08-14,M02,EMPLOYEE
E05,C1,Esha,esha@alder.example,NORTH,MON-FRI,2026-08-10,2026-08-21,M01,EMPLOYEE
E06,C1,Farid,farid@alder.example,NORTH,MON-FRI,2025-11-03,,M01,EMPLOYEE
E08,C1,Kiran,kiran@alder.example,NORTH,MON-FRI,2021-06-01,2026-07-20,M01,EMPLOYEE
M01,C1,Maya,maya@alder.example,NORTH,MON-FRI,2020-01-06,,,MANAGER
M02,C1,Omar,omar@alder.example,SOUTH,MON-FRI,2019-04-01,,,MANAGER
H01,C1,Hari,hari@alder.example,SOUTH,MON-FRI,2018-09-01,,,HR_ADMIN
B01,C2,Ivan,ivan@birch.example,COAST,MON-FRI,2024-03-01,,B02,PAYROLL_VIEWER
B02,C2,Jaya,jaya@birch.example,COAST,MON-FRI,2017-01-01,,,HR_ADMIN
```

ATTENDANCE, AUGUST 2026

One character per day, from 1 to 31 August: `P` present, `A` absent, `L` paid leave, `U` unpaid leave, `H` half day, `.` no record.

```
E01 ..PPLAP..PPPPP..P..PP..PP.PP..P
E02 P.PPPPPHPPPPPP..PPPPPP.PP.PPP.P
E03 ..............PPPPP..PPPP....
E04 ..PPPUP..PPPP..................
E05 .........PPUPP..P.PPP..........
E06 ............................
E08 ............................
M01 ..PPPPP..PPPPP..PPPPP..PP.PP..P
M02 ..PPPPP..PPPPP..PPPPP..PPPP...P
H01 ..PPPPP..PPPPP..PPPPP..PPPP...P
B01 ..PAPPP..PP.PP..PPPPP..P.PPP..P
B02 ..PPPPP..PP.PP..PPPPP..PPPPP..P
```

6 Rules

1. **Window.** For a given month, an employee's window runs from the later of the 1st of the month and `joined_on`, to the earliest of the month's last day, `left_on` and today in the company's timezone. Employees whose window is empty are left out.
2. **Scheduled working day:** a date in the window whose weekday is in the employee's roster (`MON-FRI` or `MON-SAT`) and which is not a holiday in their region.
3. **Unaccounted day:** a scheduled working day with no attendance record.
4. **Unpaid days** = the number of `A` + the number of `U` + 0.5 × the number of `H`, counted inside the window.
5. **Payable days** = calendar days in the window − unpaid days − unaccounted days. Unaccounted days are subtracted only when the company's `deduct_unaccounted` is true.
6. **At most one attendance record per employee per date**, even when requests arrive at the same moment. Records may exist on non-scheduled days (overtime); they never create or remove a gap.
7. **Who sees and records whom** follows section 4.2.

7 Part A: build

7.1 API

- `GET /summary?month=YYYY-MM`, optionally with `&onlyGaps=true`. It returns one row per employee the caller can see, with:

- `employeeId`, `name`, `region`;
- `calendarDays`, `scheduledDays`, `paidLeaveDays` (the number of `L`), `unpaidDays`;
- `unaccountedDays`, `unaccountedDates` (sorted) and `payableDays`.

With `onlyGaps=true` it returns only rows with at least one unaccounted day. A malformed month, or a month after the current one, returns 400.

- `POST /attendance` with `{ "employeeId", "date", "status" }` records one entry. It returns:
  - 201 on success;
  - 409 if a record already exists for that employee and date, including when two requests arrive at the same moment;
  - 400 for an invalid status, a date outside the employee's window, or a future date;
  - 403 or 404 as section 4.2 says.
- `GET /me` returns the signed-in user, their company and their permissions.

7.2 Frontend

- Sign-in that starts from a work email, and sign-out.
- A monthly summary with:
  - a month picker;
  - a clear flag on rows with gaps, not colour alone;
  - an "only gaps" filter;
  - the missing dates for a row, with a way to record a status for a date where the user is allowed to.
- Sensible loading, empty and error states. Include 400, 403 or 404, 409 (someone else recorded that date first), server errors and a lost connection.

> We'll use your app the way our customers do, so build it for:
>
> - large companies: 2,000 employees in one list;
> - people who move back and forth between months;
> - people who work for two companies and keep both open in two tabs of the same browser.
>
> We'll look at how fast it feels, what it caches, what it keeps in browser storage, and whether the two tabs stay independent. How you achieve that is up to you; explain your choices in `NOTES.md`.

7.3 Performance in your build

- Include a script that generates 2,000 employees with a month of attendance.
- Measure at least 100 warm requests each for `GET /summary` (as an HR admin) and `POST /attendance`, and report p50 and p95. The targets on a laptop are a p95 under 500 ms for the summary and under 100 ms for a write.
- Report your frontend's production bundle size and one page-load measurement, such as a Lighthouse run.

7.4 Tests

- The rules, using the data above.
- Access: what each role can see and record, including attempts across companies.

- Sign-in: refuse every token that comes from the wrong issuer, is meant for another app, has expired, belongs to an unknown email or a leaver, or carries an email outside the issuer's company.
- Concurrency: two identical `POST /attendance` requests sent in parallel produce exactly one 201.
- At least one frontend test.

8 Part B: DESIGN.md

Assume this load:

- **Today:** 300 companies and 60,000 employees; about 70,000 attendance writes a day, mostly between 09:00 and 11:00 IST; about 20,000 summary views a day, peaking at 50 a second in the first two working days of each month.
- **In two years:** ten times all of that.
- **Constraints:**
  - API Gateway stops waiting for a response after 29 seconds.
  - The Lambdas run in a VPC to reach Postgres.
  - Each company's data and identity must stay isolated.
  - One company's month-end rush must not slow down the others.

In at most four pages plus one diagram, cover the following. Bullet points are fine.

1. **Architecture:** the AWS services you would use and the request flow from the browser to the database.
2. **Tenant isolation:** how data, identity and access stay separate, and how you would onboard a new company with its own identity provider.
3. **RBAC at scale:** custom roles, where permissions are checked and cached, and how quickly a revoked permission takes effect.
4. **The database at ten times today's load:** schema, keys and indexes, partitioning, pre-aggregated or materialised summaries, read replicas, data retention, and migrations without downtime.
5. **Latency:**
   - a budget for `GET /summary` today and at ten times the load, showing where each millisecond goes;
   - cold starts and connection pooling;
   - caching layers, with tenant-safe keys and invalidation.
6. **Cost:** a monthly estimate for today and for ten times the load, with your assumptions. Name two changes that would cut the cost by at least 30% without hurting p95 latency.
7. **Operations:** the alarms you would set, including one that tells you a company has stopped recording attendance, and what users see when one company's identity provider is down.

9 Part C: NOTES.md and your repository

- **A git repository:** a private repository we can read, or a zip that includes the `.git` folder.
- **`README.md`:** how to run everything in a few commands, the test users, and how to sign in as each of them.
- **`DESIGN.md`:** Part B.
- **`NOTES.md`**, with:
  - your assumptions, decisions and trade-offs;
  - what is unfinished, and what you would do next;

- time spent;
- your performance results and how you measured them;
- a **browser-storage inventory**: everything your app keeps in localStorage, sessionStorage, IndexedDB, cookies or memory. Say why, for how long, and what happens to it on sign-out, when a token expires, and in other tabs;
- an **AI-use note**: which tools you used, for what, and one suggestion you changed or rejected.

10 Optional extras

Only if you have time left. They are never needed to pass.

- **Infrastructure as code** (AWS SAM, CDK or Terraform) for part of your design, validated locally and never deployed.
- **Month lock:** HR can lock a month. After that, recording attendance for it returns 409, even for requests already in flight.
- **CSV export** to S3-compatible storage, with a short-lived download link.
- **A CI workflow** that runs your tests.

11 How we evaluate

AREA	WHAT WE LOOK FOR
Correctness	The rules, on the data provided and on edge cases
Security	Tenant isolation, token validation, and permissions enforced on the server
Data design	Schema, constraints and indexes, and a credible path to ten times the load
Performance and cost	Measured numbers, efficient queries, caching that is both fast and correct, and a reasoned cost estimate
Frontend quality	Speed with large lists, caching, careful use of browser storage, independent tabs, and error states
Testing	Tests that would catch real mistakes, and honest reporting of results
Ownership and communication	A clear README, `DESIGN.md` and `NOTES.md`, and how you explain and change your work in the live session

We don't grade pixel-perfect UI, framework trivia, or optional extras you didn't finish. A polished demo doesn't make up for incorrect results.

12 The live session

About 75 minutes, on a video call.

MINUTES	WHAT HAPPENS
10	**Demo:** sign in to both companies in two tabs, show how roles differ, and fill a gap
15	**Walkthrough:** we ask about specific parts of your code
20	**Live change:** a small feature or rule change, with a test
10	**Debugging:** a failing case or a log excerpt
15	**Design:** we discuss your `DESIGN.md`: scale, latency, cost and isolation
5	**Your questions**

Use your own machine and editor. Please don't use AI assistants during the session; documentation and web search are fine.

13 Submitting and questions

- Send your repository link or zip to the recruiter who shared this assignment, by the date they gave you.
- Questions: email your recruiter, and we'll reply within one working day.
- If you're blocked, write down your assumption in `NOTES.md` and carry on. A reasonable assumption is never held against you.
- Use only the made-up data in this document, never real personal data.

Thank you for your time. We read every submission carefully and will discuss it with you in the live session.